

Context Runtime Engine for Coding Agents

Specifica completa di prodotto e architettura

1. Executive summary

L'idea è costruire un **runtime locale per agenti di coding** che si mette tra il repository e il modello usato dall'utente, con l'obiettivo di ridurre drasticamente il rumore nel contesto, migliorare la qualità del reasoning e abbassare il consumo di token. Il prodotto **non è una webapp**, non richiede hosting, non offre inferenza pesante lato server e non sostituisce il modello: l'utente continua a usare Claude Code, Codex CLI, OpenCode o altri strumenti CLI con i propri provider e piani. La parte da costruire è il **motore di contesto**, installabile localmente, con integrazione via CLI wrapper, hook e MCP dove disponibile. Claude Code supporta esplicitamente personalizzazioni tramite hooks, skills e MCP; Codex CLI è un coding agent locale open source in Rust; OpenCode è un coding agent open source con CLI/TUI e supporto per esecuzione programmatica di comandi.

In una frase:

Un "Context Runtime Engine" che seleziona, comprime, struttura e memorizza il contesto realmente utile per i coding agents, senza rompere il workflow locale dell'utente.

2. Problema che il progetto risolve

Negli strumenti di coding AI il problema non è solo la finestra di contesto massima, ma la **qualità del contesto passato al modello**. Nella pratica gli utenti incollano stack trace completi, output enormi di test/build, directory intere e file quasi irrilevanti. Questo peggiora focus e costo: il modello riceve tanto testo, ma non necessariamente le informazioni giuste.

Il progetto nasce quindi per risolvere quattro problemi insieme:

- Context bloat**: troppi token spesi in log, boilerplate, file non rilevanti.
- Context dilution**: informazioni importanti sommerse da rumore.
- Poor repository navigation**: il modello vede testo grezzo, non la struttura logica del progetto.
- Statelessness**: ogni sessione riparte quasi da zero e ricostruisce la stessa conoscenza.

Qui entra il secondo pilastro del progetto: oltre al pruning, il sistema mantiene un **Local ML/Code Knowledge Graph** che conserva struttura, simboli, relazioni e memoria operativa del repository, così l'agente può recuperare fatti rilevanti senza reiniettare continuamente grandi file markdown. MCP è proprio pensato per collegare applicazioni AI a strumenti, dati e workflow esterni in modo standardizzato.

3. Obiettivi di prodotto

3.1 Obiettivi principali

- Ridurre il contesto inviato al modello del **60–90% nei casi rumorosi**.
- Migliorare la qualità media delle risposte nei task di debugging, refactoring e repository understanding.
- Funzionare **100% in locale** su laptop comuni.
- Non richiedere GPU dedicate né servizi cloud proprietari.
- Essere installabile come utility per workflow CLI esistenti.
- Offrire un'architettura aperta per plugin e adapter.

3.2 Non-obiettivi

- Non è un nuovo LLM.
- Non è un IDE.
- Non è un orchestratore cloud.
- Non è una piattaforma SaaS di observability.
- Non vuole fare inferenza di modelli pesanti lato server.

4. Posizionamento corretto

Il progetto non va venduto come "prompt compressor". Quella categoria esiste già in ricerca e in alcuni tool sperimentali. Il posizionamento più forte è:

Nome categoria

Context Runtime Engine for Coding Agents

Messaggio chiave

"Passa al modello meno testo, ma più segnale."

Alternative di naming

- `ctxd` — Context Daemon
- `prism` — Project Relevance & Intelligent State Manager
- `codectx` — Code Context Runtime
- `context-engine`
- `agentctx`
- `prunegraph`

`codex`, `Claude Code` e `OpenCode` oggi sono già usati come agenti locali o terminal-based: questo rende molto forte un posizionamento come layer infrastrutturale orizzontale, non come prodotto concorrente.

5. User personas

Persona A — ML/AI engineer

Lavora con repository Python, notebook convertiti, script di training, log di esperimenti. Vuole dare all'agente solo il contesto utile per fixare una pipeline o rifattorizzare una feature.

Persona B — OSS maintainer

Ha codebase medio-grandi, PR complesse, molte issue. Vuole usare Claude Code/OpenCode/Codex senza sprecare contesto su cartelle irrilevanti.

Persona C — studente/ricercatore

Usa strumenti CLI e modelli anche gratuiti. Vuole massimizzare qualità e token efficiency senza pagare servizi extra.

6. Product vision

Il sistema è composto da due macro-sottosistemi:

1. Context Pruner & Optimizer

- Filtra log, diff, stacktrace e file.
- Costruisce una view compressa ma semanticamente coerente del repository.
- Espone al modello solo ciò che è utile rispetto alla query corrente.

2. Local Knowledge Graph

- Mantiene in locale una rappresentazione strutturata del progetto: file, simboli, dipendenze, ownership, hot paths, error history, task memory.
- Permette retrieval mirato senza reiniettare tutto il contenuto grezzo.

Il cuore del prodotto è l'unione di queste due cose: **compressione adattiva + memoria strutturata locale**.

7. Core workflow

7.1 Flusso base

1. L'utente esegue un comando tipo:

```
ctx ask "fix the failing pytest in auth"
```

oppure:

```
ctx wrap codex --prompt "refactor the data loader"
```

2. Il runtime:

- legge la query;
- individua segnali locali rilevanti;
- filtra output rumorosi;
- costruisce una snapshot del contesto;
- interroga il knowledge graph;
- produce un pacchetto di contesto compatto.

3. Il pacchetto viene passato all'agente scelto.

4. Dopo la risposta o l'azione dell'agente, il runtime aggiorna:

- cache;
- knowledge graph;
- statistiche di token reduction;

- traccia delle decisioni di pruning.

8. Modalità di integrazione

Il progetto deve supportare più modalità, perché i tool CLI non si integrano tutti allo stesso modo.

8.1 Wrapper mode

L'utente lancia il proprio agent attraverso `ctx`.

Esempi:

```
ctx codex "fix this stack trace"
ctx claude "explain why this test is flaky"
ctx opencode run "implement caching for embeddings"
```

Questa è la modalità più semplice e universale.

8.2 Pipe/filter mode

Il tool compatta l'output di comandi rumorosi.

```
git diff | ctx prune diff
pytest -q 2>&1 | ctx prune logs
npm install 2>&1 | ctx prune logs
```

8.3 MCP server mode

Dove disponibile, il progetto espone un **MCP server locale** con tools/resources come:

- `get_relevant_context`
- `project_map`
- `search_symbols`
- `related_failures`
- `recent_decisions`
- `get_compact_diff`

Questa modalità è molto forte perché Claude Code supporta MCP per collegare tool e dati esterni, e la specifica MCP definisce proprio risorse e tool standardizzati.

8.4 Hook mode

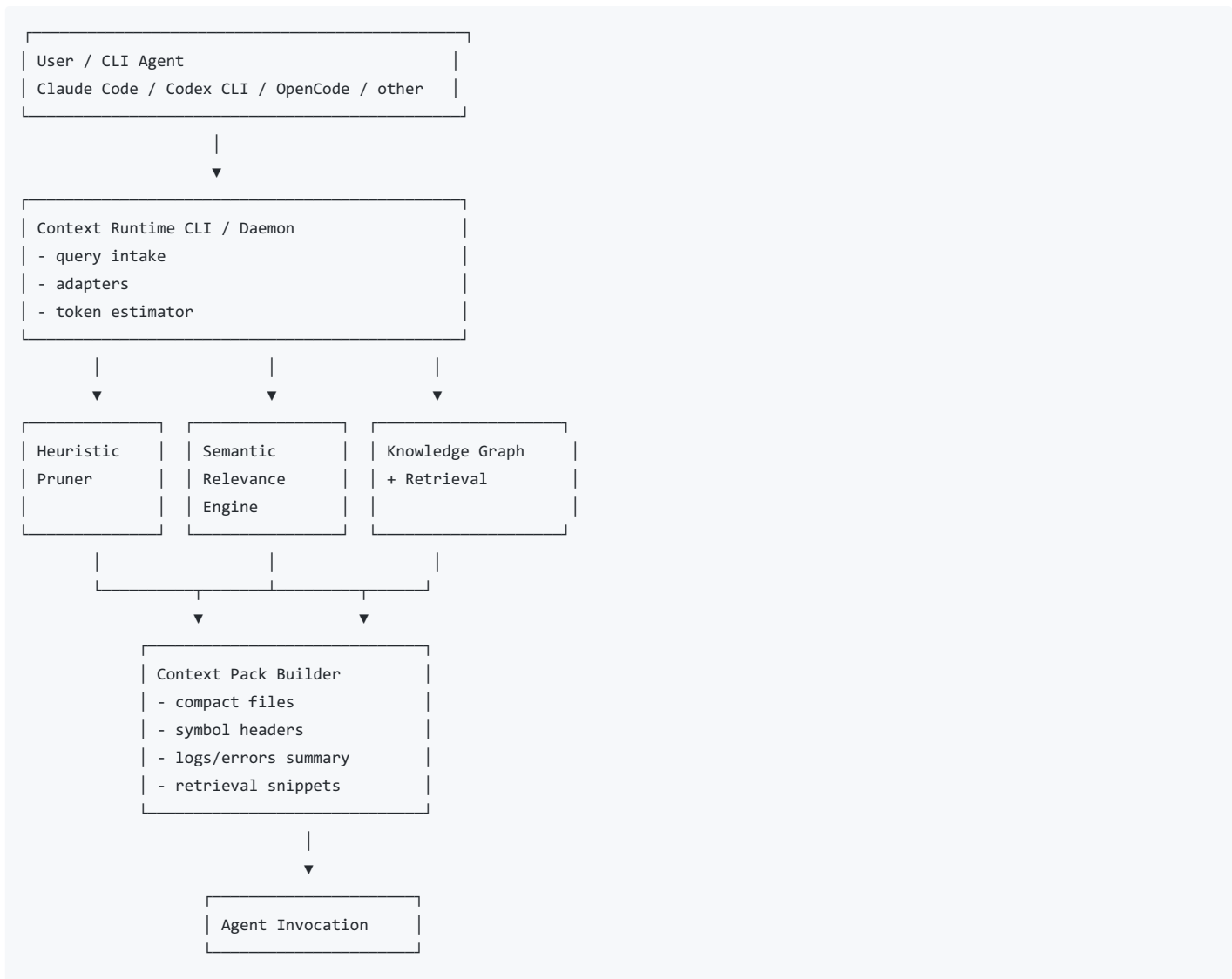
Per strumenti che permettono hook/script di pre-processing, `ctx` può essere richiamato prima della costruzione del prompt. Claude Code documenta hooks e settings gerarchici a livello utente/progetto.

8.5 Batch/index mode

Indicizzazione offline del repository:

```
ctx index
ctx reindex src tests
ctx graph rebuild
```

9. Architettura high-level



10. Componenti principali

10.1 Query Intake Layer

Responsabile di ricevere:

- prompt utente;
- eventuale comando originale;
- file selezionati manualmente;
- storico recente della sessione;
- errore o output associato.

Output: una struttura normalizzata tipo:

```
{
  "task": "fix failing pytest in auth",
  "intent": "debug",
  "repo_root": ".",
  "signals": {
    "recent_files": ["src/auth.py", "tests/test_auth.py"],
    "command_output": "...",
    "git_status": true
  }
}
```

10.2 Heuristic Pruner

Primo filtro, super veloce e deterministico.

Funzioni:

- rimuovere linee duplicate;
- sopprimere righe di successo ripetitive;
- estrarre blocchi critici da stderr;
- accorciare diff mantenendo hunk rilevanti;
- identificare stacktrace root cause;
- ignorare build noise noto.

Esempi:

- 10k righe di `npm install` → tieni warning/error e riepilogo package.
- `pytest` con 500 test passati e 1 fallito → tieni il fallimento, il traceback e i test dipendenti.
- `git diff` enorme → tieni file modificati, firme, hunk attinenti alla query.

10.3 Syntax & Structure Analyzer

Qui entra `tree-sitter`. Tree-sitter è progettato per generare syntax tree incrementali e aggiornabili efficientemente, quindi è perfetto per indexing e slicing di codice locali.

Funzioni:

- parsing incrementale multi-linguaggio;
- estrazione di simboli: classi, funzioni, import, metodi, test;
- relazione file → simboli;
- slice strutturali senza rompere il codice.

10.4 Semantic Relevance Engine

Secondo filtro, query-aware.

Funzioni:

- embedding leggero della query;
- embedding dei blocchi di codice e dei documenti tecnici;
- similarity ranking;
- soglie adattive;
- deduplicazione semantica.

Per l'inferenza locale CPU conviene usare ONNX Runtime, che applica ottimizzazioni sul grafo e offre kernel ottimizzati ed execution providers multiplatforma. Sentence Transformers supporta anche backend/export ONNX.

10.5 Context Rewriter

Non si limita a eliminare testo: lo **riscrive in forma compatta**.

Esempi:

- firma completa + doc breve invece di 200 righe.
- diff compatto con campi strutturati.
- summary gerarchico per moduli lunghi.

Esempio:

```
Original file length: 1,842 lines
Compressed view:
- AuthService: token issuance, refresh validation, session revocation
- Failing path touches: validate_refresh_token -> decode_token -> key resolver
- Related tests: test_refresh_expired_token, test_refresh_wrong_audience
```

10.6 Knowledge Graph Engine

Questo è il componente che rende il progetto più forte e meno facilmente copiabile.

Entità principali

- repository
- directory
- file
- symbol
- import/dependency
- test
- command run
- failure
- task
- decision
- note
- issue/PR reference locale

Archi/relazioni

- contains
- imports
- calls
- defines
- tests
- failed_in
- related_to_query
- edited_with
- mentioned_in_task

Esempio

`src/auth.py::validate_refresh_token`

- is_defined_in → `src/auth.py`
- is_tested_by → `tests/test_auth.py::test_refresh_expired_token`
- imported_by → `src/api/routes/auth_routes.py`
- last_failed_in → `pytest run #184`
- related_to_task → `fix expired refresh token bug`

10.7 Retrieval Layer

Fa retrieval ibrido:

- graph traversal;
- keyword/FTS;
- embedding similarity;
- recency bias.

SQLite è molto adatto come base locale perché supporta FTS5 per full-text search e JSON functions/JSON1 per dati strutturati embedded, senza richiedere servizi esterni.

10.8 Adapters Layer

Adapter specifici per:

- Claude Code
- Codex CLI
- OpenCode
- generic CLI provider
- futura estensione per Aider / Gemini CLI / altri agenti terminal-based

11. Pipeline completa di contesto

Stage 0 — Signal collection

Raccoglie:

- query;
- repo root;
- git status/diff;
- ultimo comando fallito;
- file toccati di recente;
- test falliti;
- session memory.

Stage 1 — Fast deterministic pruning

Regole statiche e parser specifici per comando.

Stage 2 — Structural slicing

Usa AST e symbol map per creare blocchi candidati.

Stage 3 — Semantic ranking

Ordina i blocchi rispetto alla query.

Stage 4 — Graph enrichment

Aggiunge test correlati, dipendenze immediate, failure storiche e decisioni passate.

Stage 5 — Context budget packing

Compone il contesto finale rispettando un budget configurabile.

Stage 6 — Invocation and telemetry

Passa il contesto all'agente e registra metriche locali.

12. Context packing strategy

Ogni prompt riceve un budget esplicito, per esempio 6k token. Il packer deve decidere cosa includere.

Ordine di priorità suggerito

1. query utente;
2. error root cause;
3. simboli direttamente rilevanti;
4. test associati;
5. diff recente;
6. dipendenze immediate;
7. memoria/decisioni pregresse;
8. documentazione secondaria.

Garanzie minime

Anche quando un blocco viene compresso, devono essere preservati:

- path del file;
- firma del simbolo;
- import significativi;
- relazione con altri simboli;
- range di linee originali o riferimenti strutturali.

13. Esempi d'uso concreti

13.1 Debug di test Python

```
pytest tests/test_auth.py -q 2>&1 | ctx prune logs > /tmp/auth_fail.txt
ctx codex "Fix this failure using the relevant auth code" --attach /tmp/auth_fail.txt
```

Output atteso di `ctx` prima dell'invio:

```
Context reduction: 14,822 -> 2,116 tokens (-85.7%)
Included:
- failing traceback root cause
- src/auth.py::validate_refresh_token
- src/auth.py::decode_token
- tests/test_auth.py::test_refresh_expired_token
- recent diff hunks in auth module
Excluded:
- 213 passing tests
- unrelated fixtures
- cache/build noise
```

13.2 Refactor mirato su repository grande

```
ctx claude "Refactor the data loader to separate parsing from feature engineering"
```

Il sistema:

- individua file che contengono `DataLoader`, `parse`, `feature_engineering`;
- aggiunge dipendenze immediate;
- esclude cartelle come `notebooks/`, `artifacts/`, `build/`, `dist/`.

13.3 Context map senza invocare un agente

```
ctx explain "Where is retry logic implemented?"
```

Risposta locale:

```
Likely relevant symbols:
1. src/http/client.py::retry_request
2. src/http/backoff.py::exponential_backoff
3. tests/test_retry.py::test_retries_on_429
Related command history:
- failing integration test on 2026-04-23
```

13.4 Uso come MCP server

```
ctx mcp serve
```

Da Claude Code l'agente può poi richiamare tools/resources del server locale per chiedere contesto aggiuntivo senza incollare file manualmente. Claude Code documenta esplicitamente l'uso di MCP per connettere tool e sorgenti esterne.

13.5 Uso con OpenCode

```
opencode run "Explain this failing TypeScript build" --output text \
| ctx prune logs \
| tee compact-build.txt
```

OpenCode documenta una CLI che può essere usata anche in modo programmatico, non solo via TUI. [\[1\]](#)

13.6 Uso con Codex CLI

```
ctx codex "Review the last diff and find risky changes"
```

Codex CLI è documentato come coding agent locale open source che legge, modifica ed esegue codice sul computer dell'utente, quindi un wrapper locale è una modalità naturale di integrazione. [\[2\]](#)

14. CLI design proposta

Comandi top-level

```
ctx init
ctx index
ctx reindex
ctx graph build
ctx graph query "where is token refresh logic"
ctx prune logs
ctx prune diff
ctx pack "fix this failing test"
ctx explain "why is this module relevant"
ctx codex "..."/>

```

Esempio `ctx pack`

```
ctx pack "fix flaky login test" --budget 6000 --json
```


Output:

```
{
  "original_estimated_tokens": 23814,
  "packed_tokens": 4172,
  "reduction_pct": 82.48,
  "included": [
    "tests/test_login.py::test_session_timeout",
    "src/auth/session.py::refresh_session",
    "src/auth/tokens.py::decode_token"
  ],
  "excluded": [
    "node_modules/",
    "build logs",
    "unrelated docs"
  ],
  "pack_path": ".ctx/packs/pack_184.json"
}
```

15. Knowledge Graph: specifica tecnica

15.1 Perché serve davvero

La memoria come semplice file markdown ha tre problemi:

- cresce in fretta;
- spreca token;
- non è interrogabile strutturalmente.

Il knowledge graph risolve questo perché conserva **fatti e relazioni**, non solo testo libero.

15.2 Storage design

Scelta consigliata: **SQLite + tabelle normali + FTS5 + JSON columns**.

Tabelle possibili

- files
- symbols
- edges
- tasks
- runs
- failures
- notes
- snippets
- embeddings_metadata

Perché non Neo4j

- richiede runtime separato;
- rompe il vincolo “0 euro / facile da installare”;
- peggiora onboarding.

15.3 Query utili

```
-- simboli collegati a un test fallito
SELECT s.name, f.path
FROM symbols s
JOIN edges e ON e.dst_symbol_id = s.id
JOIN files f ON f.id = s.file_id
WHERE e.type = 'tests'
      AND e.src_symbol_name = 'test_refresh_expired_token';
```

Oppure via CLI:

```
ctx graph query "show auth symbols related to refresh failures"
```

15.4 Dati memorizzati

- simboli estratti via AST;
- riferimenti import/call;
- ultimo editing timestamp;
- ultimi task che li hanno toccati;
- failure note;
- snippet compressi;
- embedding metadata.

15.5 Policy di aggiornamento

- file modified → reparse incrementale;
- commit nuovo → refresh dipendenze;
- comando fallito → crea nodo `failure`;
- task concluso → salva decision summary breve.

16. Semantic pruning: strategia consigliata

16.1 Input units

Segmentare il codice in unità significative:

- modulo;
- classe;
- funzione/metodo;
- test case;
- config block;
- doc chunk.

16.2 Ranking features

Punteggio ibrido:

- similarità semantica query ↔ chunk;
- overlap keyword/symbol names;
- prossimità al diff recente;
- vicinanza nel call graph;
- recency;
- failure relevance.

Formula semplice iniziale

```
score = 0.40 * semantic_similarity
       + 0.20 * keyword_overlap
       + 0.15 * recency
       + 0.15 * graph_distance_bonus
       + 0.10 * failure_bonus
```

16.3 Safe pruning rules

Non tagliare mai automaticamente:

- import critici;
- firme pubbliche;
- class headers;
- decorator rilevanti;
- docstring di API pubbliche quando spiegano comportamento.

17. Log pruning: parser specifici

Uno dei punti più virali del progetto è avere parser ottimizzati per output comunissimi.

Parser prioritari

- `pytest`
- `python traceback`
- `npm install`
- `tsc`
- `cargo build`
- `go test`
- `git diff`
- `git status`
- `ruff/mypy/eslint`

Esempio `pytest`

Input: 8,000 righe. Output compatto:

- 1 failure summary
- stack root cause
- fixture chain rilevante
- test names correlati
- ultimi file toccati

18. Stack tecnologico consigliato

18.1 Linguaggio principale

Rust

Motivi:

- performance alta;
- binario standalone;
- ottimo per CLI e daemon locali;
- packaging solido;
- allineato al posizionamento "serio" da systems tool.

Nota: anche Go sarebbe valido, ma Rust è più forte se vuoi enfatizzare parsing, memory efficiency e tooling low-level.

18.2 Librerie/tecnologie consigliate

Core CLI

- `clap` — parsing CLI
- `tokio` — async runtime dove serve
- `serde` / `serde_json` — config e payload
- `tracing` — logging strutturato

Parsing codice

- `tree-sitter`
- grammar language-specific per Python, TS/JS, Rust, Go, Java ecc.

Knowledge graph / storage

- `rusqlite`
- SQLite con FTS5/JSON1

Embeddings / relevance

- ONNX Runtime
- modello embedding piccolo quantizzato o esportato ONNX

MCP

- server MCP locale implementato in Rust

Token estimation

- tokenizer libs per stima budget

Cache

- SQLite o file cache locale in `.ctx/`

19. Configurazione

Esempio `.ctx/config.toml`:

```
[general]
repo_root = "."
default_budget = 6000
agent = "codex"

[pruning]
collapse_success_logs = true
keep_imports = true
keep_public_signatures = true
max_log_lines = 200

[semantic]
enabled = true
backend = "onnx"
model = "local-mini-embed"
max_chunks = 64

[graph]
enabled = true
store = ".ctx/graph.db"
index_tests = true
index_docs = true

[mcp]
enabled = true
port = 8765
```

20. Installazione desiderata

20.1 Utente finale

```
brew install context-engine
# oppure
cargo install context-engine
# oppure binario precompilato da GitHub Releases
```

Poi:

```
ctx init
ctx index
ctx stats
```

20.2 Integrazione con Claude Code

Possibili modalità:

- wrapper locale `ctx claude ...`
- hook di pre-processing
- MCP server registrato nel tool

Claude Code documenta settings di progetto/utente, hooks, skills e MCP, quindi l'integrazione è realistica senza costruire una UI dedicata.
[↗cite](#)[↗turn623453search0](#)[↗turn623453search8](#)[↗turn623453search12](#)

20.3 Integrazione con Codex CLI

```
ctx codex "refactor token validation"
```

Dato che Codex CLI gira localmente sul computer dell'utente, il wrapper non entra in conflitto con il modello/provider: si limita a ottimizzare il contesto prima dell'invocazione. [@turn623453search9@turn623453search17](#)

20.4 Integrazione con OpenCode

```
ctx opencode run "explain this diff"
```

Oppure `ctx` come stage separato in pipe. OpenCode documenta sia TUI sia comandi CLI utilizzabili programmaticamente.

[@turn623453search2@turn623453search14](#)

21. Sicurezza e privacy

Punto fortissimo del progetto: i dati restano locali.

Principi

- nessun upload automatico verso server del progetto;
- knowledge graph solo locale;
- telemetria opt-in e disattivata di default;
- file sensibili esclusi via ignore rules;
- audit log locale delle decisioni di pruning.

Questo è particolarmente coerente con il modello MCP, che sottolinea esplicitamente considerazioni di sicurezza e trust quando si collega un'app AI a dati e strumenti locali.

22. Metriche di qualità del progetto

Per rendere il progetto credibile su GitHub/Reddit non basta dire "riduce token". Devi mostrare metriche serie.

Metriche principali

- token reduction %
- latency overhead locale
- task success rate vs baseline
- answer quality judged by user
- fix success on benchmark repos
- retrieval precision@k per simboli rilevanti

Benchmark suggeriti

- repo Python medio-piccoli
- repo TS/JS
- repo Rust/Go
- task di debug e refactor riproducibili

23. MVP realistico

Fase 1 — MVP virale

Obiettivo: demo che fa dire "ok, questa roba mi serve".

Include

- wrapper CLI
- prune logs
- prune diff
- index
- symbol extraction via tree-sitter
- simple SQLite graph
- budget packer
- report token diff

Demo README ideale

```
Before: 42,118 tokens
After: 4,906 tokens
Reduction: 88.3%
Still included:
- failing traceback
- touched auth code
- related tests
- recent diff
```

Fase 2 — Semantic engine

- embedding locale ONNX
- ranking ibrido
- explainability del punteggio

Fase 3 — MCP + adapters seri

- MCP server
- hook integration
- config presets per Claude Code / Codex / OpenCode

Fase 4 — Knowledge graph avanzato

- failure memory
- task memory
- decision summaries
- graph query language semplice

24. Cosa rende il progetto virale

24.1 Demo immediata

Una GIF che mostra:

- log gigantesco;
- `ctx prune logs` ;
- riduzione drastica;
- agente che risolve il problema con meno contesto.

24.2 Output trasparente

L'utente deve vedere cosa è stato escluso e perché.

24.3 Nessun lock-in

Funziona con il modello che l'utente già usa.

24.4 Local-first

Zero cloud obbligatorio.

24.5 OSS contributor-friendly

Parser, adapters e language packs sono contributi facili da aprire alla community.

25. Possibili estensioni future

- supporto notebook-aware;
- supporto PR review mode;
- benchmark suite open per context quality;
- rule packs per stack specifici (PyTorch, FastAPI, React, Next.js, Rust crates);
- active learning del relevance ranking;
- export di "project capsules" condivisibili.

26. Roadmap sintetica consigliata

Week 1

- CLI base
- config
- log pruning parser `pytest`, `traceback`, `git diff`

Week 2

- tree-sitter indexing
- SQLite schema
- graph build minimo

Week 3

- context packer
- token diff report
- wrapper Codex/OpenCode/Claude generico

Week 4

- semantic ranking ONNX
- explain mode
- benchmark demo

Week 5+

- MCP server
- memory/decision layer
- community adapters

27. README pitch in una frase

A local-first context runtime for coding agents that cuts noise, preserves signal, and gives your AI the right code instead of the whole repo.

28. Conclusione

Questa idea è forte perché unisce tre cose che raramente stanno insieme nello stesso progetto:

1. **utilità immediata** per chi usa agenti di coding;
2. **difficoltà tecnica credibile** da systems/NLP tooling project;
3. **adottabilità reale** perché non obbliga nessuno a cambiare modello, IDE o provider.

La parte davvero differenziante non è soltanto comprimere testo, ma costruire un **runtime locale intelligente** che capisce struttura del repository, osserva i flussi di lavoro e conserva memoria utile in un knowledge graph interrogabile. In questo modo il progetto resta piccolo abbastanza da essere open source e locale, ma abbastanza ambizioso da sembrare un prodotto di livello alto.

Se costruito bene, questo non è solo un side project: è uno di quei repository che possono diventare riferimento pratico per chi usa coding agents ogni giorno.